Lecture Notes for

Neural Networks I

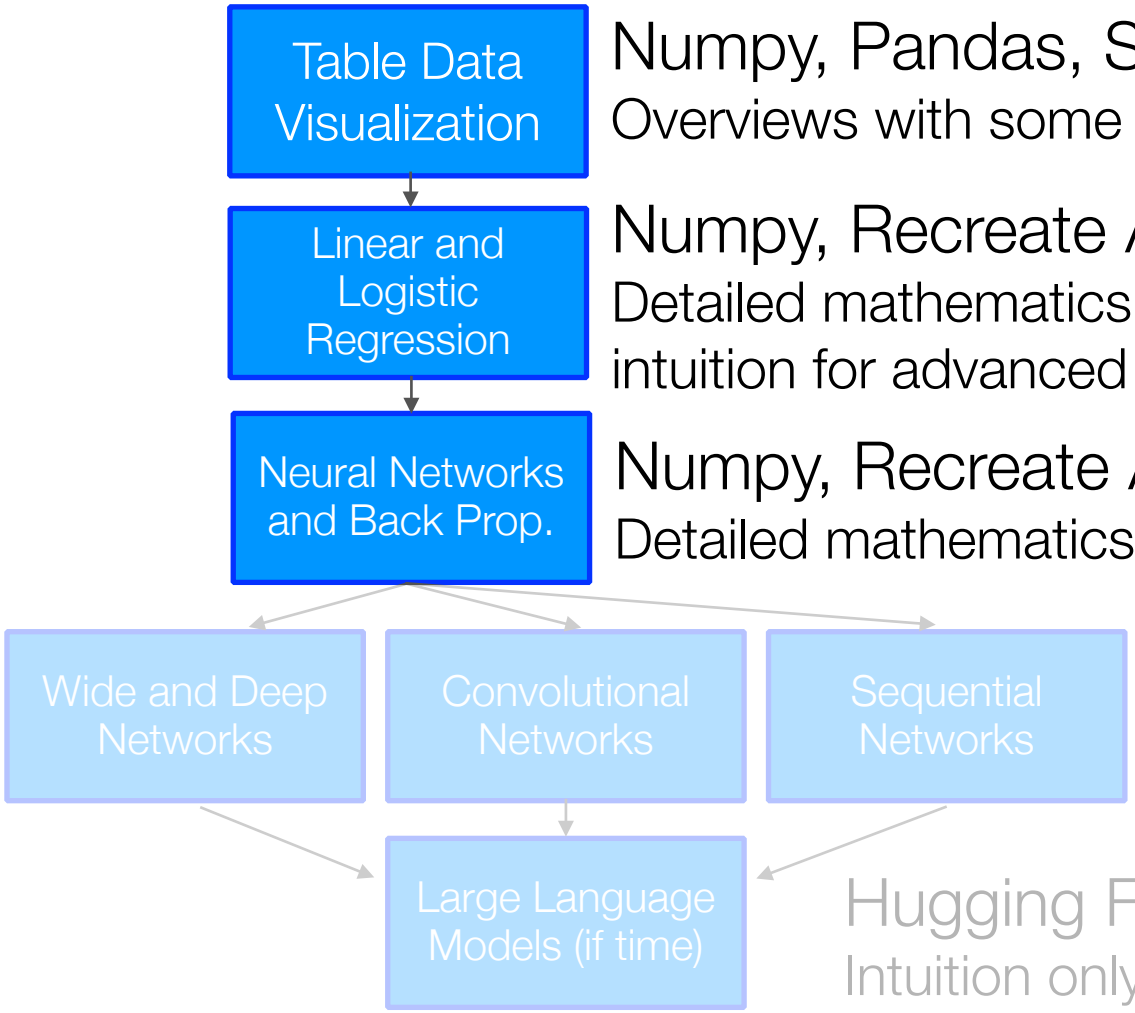


Neural Network Wrap Up, Review

Class Logistics and Agenda

- Logistics
  - Grading update
- Agenda:
  - Neural Network Review
  - Universality, Revisited
  - Mid-term Review

Class Overview, by topic



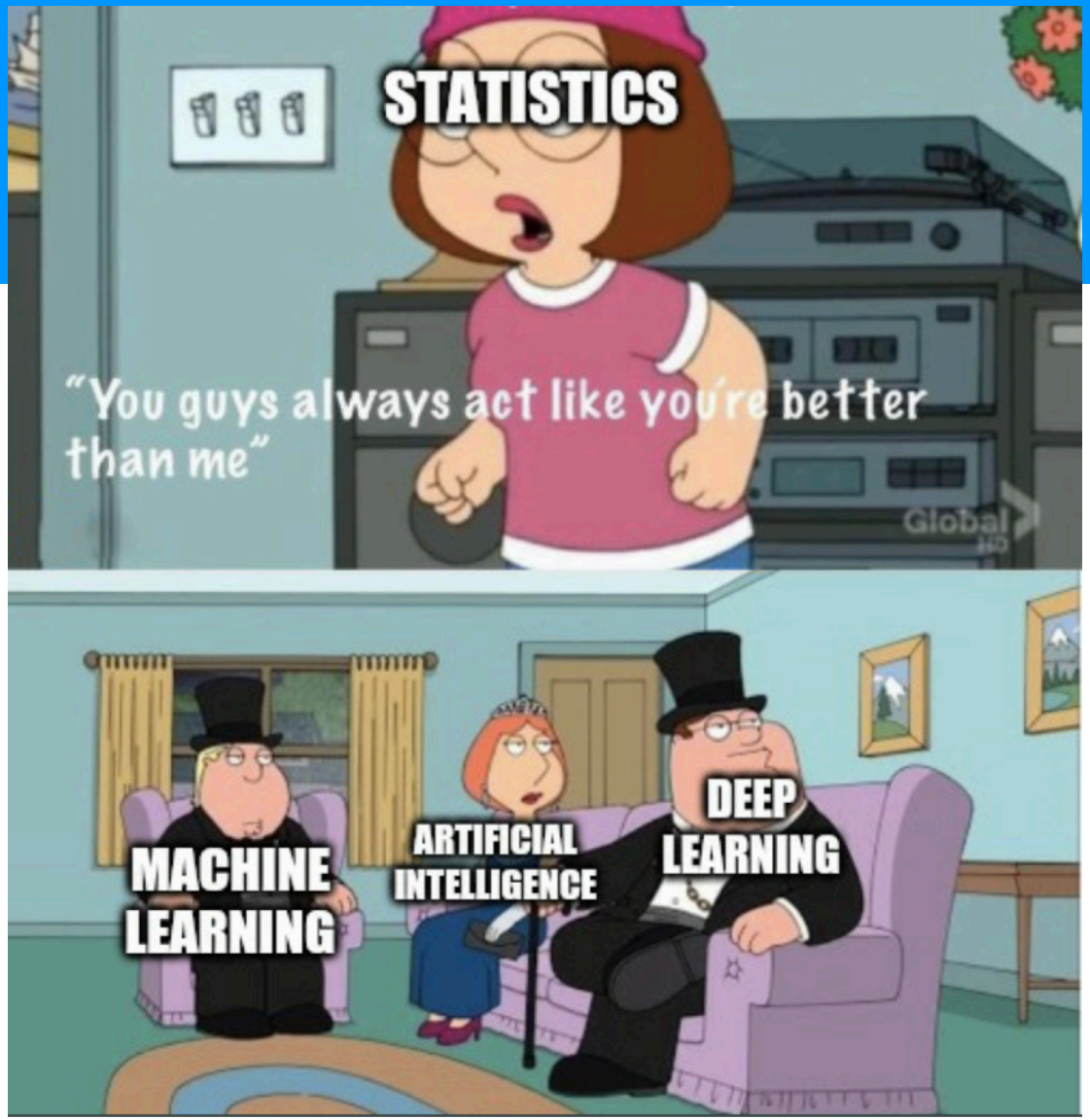
Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

Hugging Face
Intuition only

Review


STATISTICS
"You guys always act like you're better than me"
Global
MACHINE LEARNING
ARTIFICIAL INTELLIGENCE
DEEP LEARNING

Review

$$\mathbf{W}_{k+1} = \mathbf{W}_k - \rho_k$$

- Cross entropy

$$\mathbf{V}^{(2)} = \mathbf{A}^{(3)} - \mathbf{Y}$$

new final layer update

- Momentum

$$\rho_k = \alpha \nabla J(\mathbf{W}_k) + \beta \nabla J(\mathbf{W}_{k-1})$$

- Nesterov's Momentum

$$\rho_k = \underbrace{\beta \nabla J\left(\mathbf{W}_k + \alpha \nabla J(\mathbf{W}_{k-1})\right)}_{\text{step twice}} + \alpha \nabla J(\mathbf{W}_{k-1})$$

- Mini-batching

←all data→

	batch 1	batch 2	batch 3	batch 4	batch 5	batch 6	batch 7	batch 8	batch 9
Epoch 1									
Epoch 2									
Epoch 3									
Epoch 4									
...									

shuffle ordering each epoch and update W's after each batch

- Learning rate adjustment (eta)

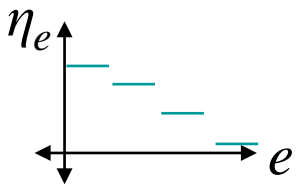
$$\eta_e = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})\left(1 + \cos\left(\frac{e}{e_{max}}\pi\right)\right)$$

Stair step

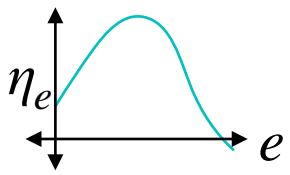
$$\eta_e = \eta_0 \cdot d^{\lfloor \frac{e}{e_d} \rfloor}$$

e_d epochs between reductions

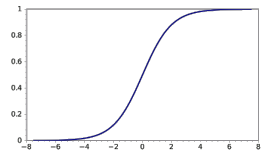
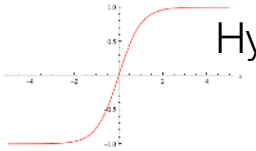
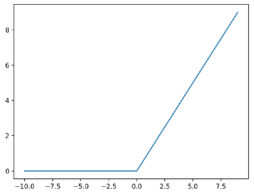
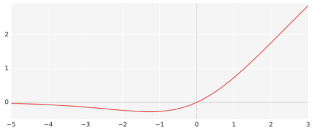
$0 < d < 1$



Cosine Schedule



Review: Activations Summary

	Definition	Derivative	Weight Init (Uniform Bounds)
 Sigmoid	$\phi(z) = \frac{1}{1+e^{-z}}$	$\nabla\phi(z) = a(1-a)$	$w_{ij}^{(L)} \sim \pm 4\sqrt{\frac{6}{n^{(L)}+n^{(L+1)}}}$ $b^{(l)} = [0]$
 Hyperbolic Tangent	$\phi(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$\nabla\phi(z) = \frac{4}{(e^{z}+e^{-z})^2}$	$w_{ij}^{(L)} \sim \pm\sqrt{\frac{6}{n^{(L)}+n^{(L+1)}}}$ $b^{(l)} = [0]$
 ReLU	$\phi(z) = \begin{cases} z, \text{ if } z > 0 \\ 0, \text{ else} \end{cases}$	$\nabla\phi(z) = \begin{cases} 1, \text{ if } z > 0 \\ 0, \text{ else} \end{cases}$	$w_{ij}^{(L)} \sim \pm\sqrt{2}\sqrt{\frac{6}{n^{(L)}+n^{(L+1)}}}$
 SiLU	$\phi(z) = \frac{z}{1+e^{-z}}$	$\nabla\phi(z) = \phi(z) + \sigma(z)\cdot(1-\phi(z))$	$b^{(l)} = [0]$

Review of Adaptive Strategies $\mathbf{W}_{k+1} = \mathbf{W}_k - \eta \cdot \rho_k$

Adjust each element of gradient by the steepness (for each layer):

- AdaGrad

$$\rho_k = \frac{1}{\sqrt{\mathbf{G}_k + \epsilon}} \odot \nabla J(\mathbf{W}_k)$$

all operations are per element

where

$$\mathbf{G}_k = \gamma \cdot \mathbf{G}_{k-1} + \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

- RMSProp

$$\rho_k = \frac{1}{\sqrt{\mathbf{V}_k + \epsilon}} \odot \nabla J(\mathbf{W}_k)$$

all operations are per element

$$\mathbf{G}_k = \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

$$\mathbf{V}_k = \gamma \cdot \mathbf{V}_{k-1} + (1-\gamma) \cdot \mathbf{G}_k$$

- AdaDelta

$$\rho_k = \frac{\mathbf{M}_k}{\sqrt{\mathbf{V}_k + \epsilon}}$$

all operations are per element

$$\mathbf{M}_{k+1} = \gamma \cdot \mathbf{M}_k + (1-\gamma) \cdot \nabla J(\mathbf{W}_k)$$

- AdaM

update momentum

$$\mathbf{M}_{k+1} \leftarrow \beta_1 \cdot \mathbf{M}_k + (1-\beta_1) \cdot \nabla J(\mathbf{W}_k)$$

$$\hat{\mathbf{M}}_k \leftarrow \frac{\mathbf{M}_k}{(1-[\beta_1]^k)}$$

normalizer momentum

$$\mathbf{V}_{k+1} \leftarrow \beta_2 \cdot \mathbf{V}_k + (1-\beta_2) \cdot \nabla J(\mathbf{W}_k) \odot \nabla J(\mathbf{W}_k)$$

$$\hat{\mathbf{V}}_k \leftarrow \frac{\mathbf{V}_k}{(1-[\beta_2]^k)}$$

exploitation, boosting

full update

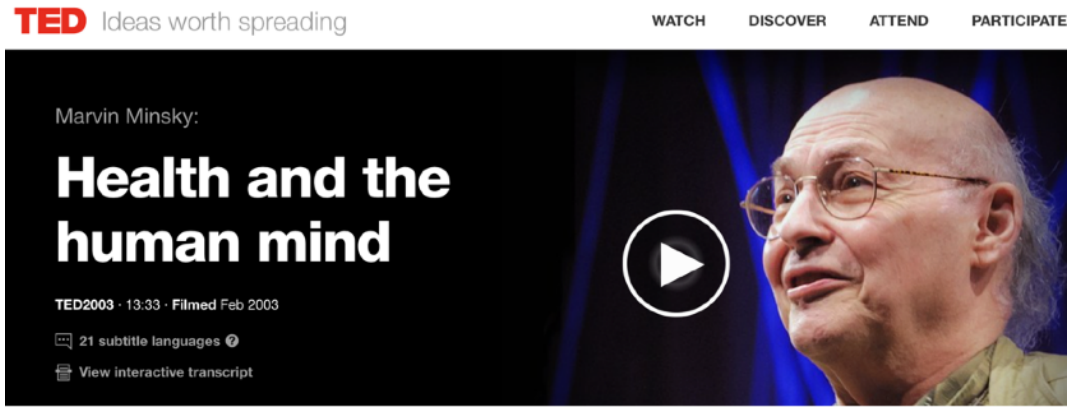
$$\mathbf{W}_{k+1} \leftarrow \mathbf{W}_k - \eta \cdot \frac{\hat{\mathbf{M}}_k}{\sqrt{\hat{\mathbf{V}}_k + \epsilon}}$$

Revisiting Universality (if time)

- Neural networks can separate any data through multiple layers. The true realization of Rosenblatt:

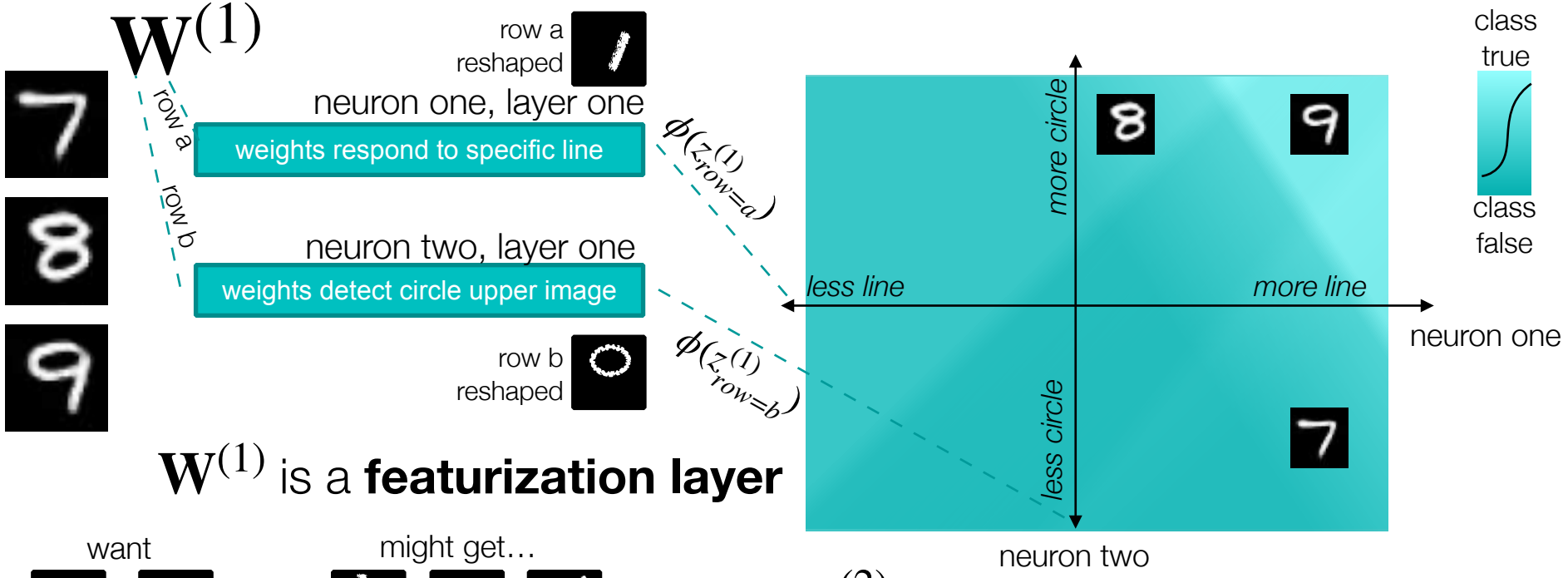
> "Given an elementary α-perceptron, a stimulus world W, and any classification C(W) for which a solution exists; let all stimuli in W occur in any sequence, provided that each stimulus must reoccur in finite time; then beginning from an arbitrary initial state, an error correction procedure will always yield a solution to C(W) in finite time…"

- **Universality**: No matter what function we want to compute, we know that there is a neural network which can do the job.





Universality



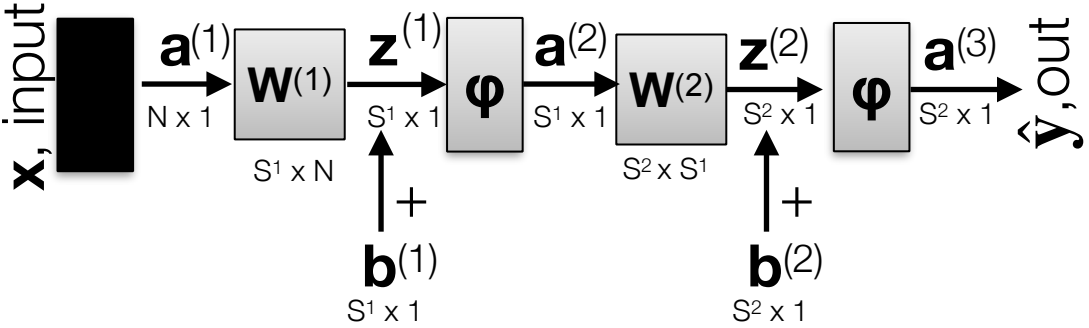
$\mathbf{W}^{(1)}$ is a **featurization layer**



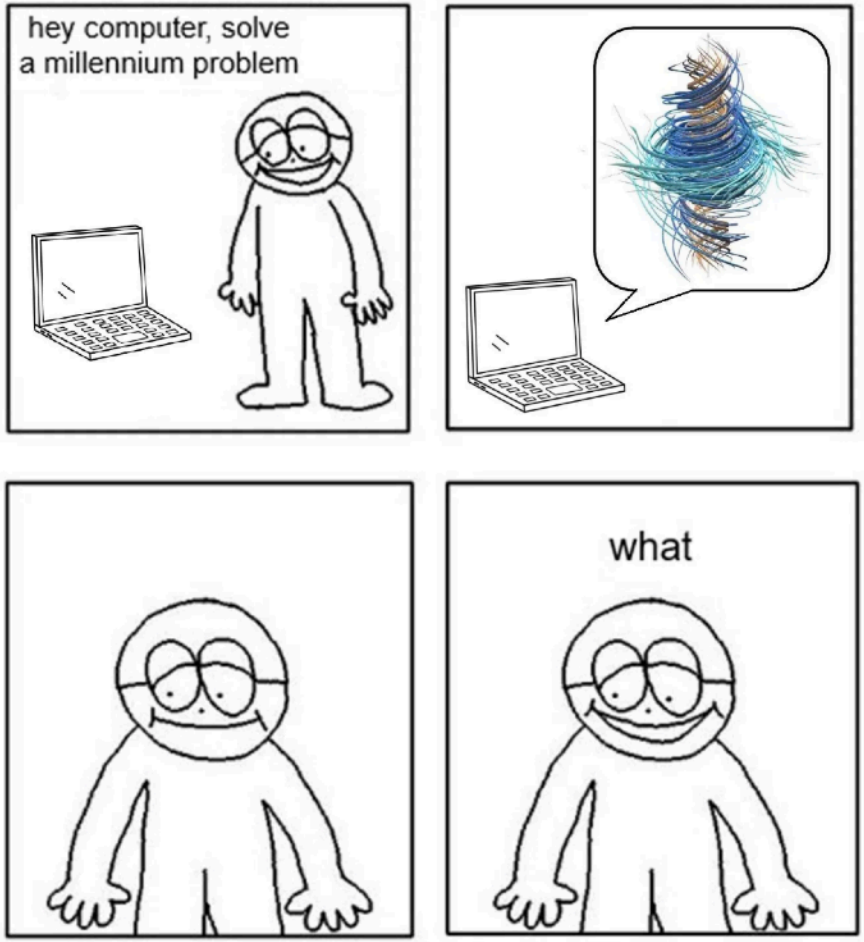
adding layers and convolution help obtain desired behavior…

$\mathbf{W}^{(2)}$ is **classification layer,** like one versus all logistic regression

- One nonlinear hidden layer with an output layer can ***perfectly train any problem with enough data***, but might be memorizing…
- … could be better to have ***even more layers*** for more generalizing features



Midterm Review



Exam Format and Possible Content

- 15 multiple choice questions (50% credit)
- 3 multi-part, open-ended questions (50% credit)
- One sheet of notes, front/back
- No phones, devices, etc.
- Table Data, Numeric/Categorical, Visualization
- Missing Data, **Imputation**, Exploratory Data Analysis
- Logistic Regression, Techniques, Interpretation, *"What if we changed… ?"* **Decision Boundaries**
- Neural Networks: Notation, Optimization, Adaptive Techniques,
- **Drawing** a described network, with chain rule markup

Example MC Questions x15

Which of the following is most desirable when selecting a dataset for a supervised classification task?

A. The dataset contains mostly numeric variables and continuous numeric labels.
B. The dataset contains both predictor variables and class labels suitable for prediction.
C. The dataset contains images with annotations.
D. The dataset has no missing values to minimize preprocessing.
E. The dataset has more features than observations.
F. None of the above.

A feature contains approximately 80% missing values that appear to be randomly distributed throughout the dataset. Which approach is generally the most reasonable first choice?

A. Delete the feature from the dataset.
B. Impute the missing values using split-impute-combine.
C. Impute the missing values using k-nearest neighbors.
D. Impute the missing values with zero.
E. All of the above.
F. Both B and C.

The primary purpose of adding regularization (such as L2 regularization) to logistic regression is to:

A. Increase the number of training examples.
B. Reduce overfitting by discouraging excessively large model coefficients.
C. Mitigate the need for input feature scaling.
D. Both A and B.
E. Both B and C.
F. None of the above

In gradient descent, model parameters are updated by moving:

A. In the direction of the gradient to maximize the loss.
B. Opposite the gradient to reduce the loss function.
C. Randomly until convergence.
D. Toward the largest feature values.

Example FRQ:

- Use KNN (k=2) with L1 distance to impute missing data in the following Table. Use mean to impute the missing value. Show distance to each instance and call out any assumptions you made in your calculation.

Age Range	***ParCh***	***Fare***	***Label***
35-40	*3*	*$5*	*Survived*
25-30	*0*	*$13*	*Survived*
40-45	*5*	*NaN*	*Perished*
20-25	*7*	*$3*	*Perished*

Example FRQ:

- Draw a diagram of a neural network with three input features, one hidden layer (3 neurons), and four classes. mark the size of all feedforward variables and annotate the gradient calculations needed to update **W** and **b** in the first layer.

Example FRQ:

- Manipulate the following logistic regression objective function such that we are optimizing log-likelihood rather than just likelihood. Show all steps and simply the equation as much as possible.

$$L(\mathbf{w}) = \prod_i g\left(\mathbf{x}^{(i)} \cdot \mathbf{w}\right)_{y^{(i)}=1} \cdot \left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)_{y^{(i)}=0}$$

Next Time

- Mid term Exam
- Then: final flipped module after the break!